

HPC School



HPC Primer - Git

September 2026

The Git PDF:
<https://edu.nl/98drq>

whoami Instructors

Gérson Fernandes

- Aerospace engineer. Now simulations engineer @supercomputing center.
- Amateur car mechanic.

Alain van Hoof

- Seasoned systems administrator. Manager of TU/e's Umbrella Cluster.
- Ice skating prodigy.

Requirements I

A SURF Cua account

<https://portal.cua.surf.nl>

A terminal on Snellius

<https://ondemand.snellius.surf.nl>

Requirements II

Basic terminal (bash) knowledge

Github Account

got to github.com

Create (Sign up) an account (if you don't have one)

Git

Distributed Version Control, successor to BitKeeper,
SubVersion (SVN), CVS, SCCS, RCS

Created by: **Linus Trovalds**

Yes! that person who wrote and is still involved in, the
Linux Kernel development.

10 Days!

Torvalds famously wrote Git in just 10 days after Linux
kernel developers
lost access to their proprietary tool, BitKeeper,
due to licensing disagreements

Linux



**My Setup programming in python
with ChatGTP**



Uploading public key -> github

To use SSH with Github, copy your public! key to your Github account.

1. Sign in to Github.
2. On the top right, select the avatar.
3. Select **Settings**.
4. On the left sidebar, select **SSH and GPG Keys**.
5. Select **new SSH key**.
6. In the **Title** box, type a description.
7. For **Key type** use the default, **Authentication**.
8. In the **Key** box, paste the contents of your public key.
9. Select **Add SSH key**.

Copy the contents of your public key file. You can do this manually or use a script. For example, to copy an ED25519 key to the clipboard:

macOS

```
tr -d '\n' < ~/.ssh/id_ed25519.pub | pbcopy
```

Linux (requires the `xclip` package)

```
xclip -sel clip < ~/.ssh/id_ed25519.pub
```

Git Bash and PowerShell on Windows

```
cat ~/.ssh/id_ed25519.pub | clip
```

Replace `id_ed25519.pub` with your filename.
For example, use `id_rsa.pub` for RSA.

Check

Verify that your SSH key was added correctly.

To ensure you're connecting to the correct server,
check the server's SSH host keys fingerprint.

For github.com, see the SSH host keys fingerprints at
<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/githubs-ssh-key-fingerprints>
(<https://edu.nl/dqbbqd>)

Open a terminal and run these commands:

```
ssh-add -L
```

if you key is not visible:

```
ssh-agent bash  
ssh-add
```

Now Check

```
ssh -T git@github.com
```

If this is the first time you connect,
you should verify the authenticity of the github host.

If you see a message like:

```
The authenticity of host 'github.com (140.82.121.3)' can't be established.  
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvC0qU.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Type **yes** (not just y) and press **Enter**.

Run the

```
ssh -T git@github.com
```

command again. You should receive a

```
Hi `@username`! You've successfully authenticated, but GitHub does not provide shell access.
```

message.

If the welcome message doesn't appear, you can
troubleshoot
by running `ssh` in verbose mode:

```
ssh -Tvvv git@github.com
```

Why version control

Git & HPC

- `git` is used across the computing industry for different reasons
- We are introducing it here because often researchers work on at least 2 systems:
 - Their own laptop
 - The remote HPC cluster

HPC & Git

- Often users do not keep track of what they changed
- Nightmare scenarios can be avoided:
 - "Which version of my code did I actually run? The one with or without the bug?"
 - Why am I getting different results? Did I change my input file?"
 - "My computer crashed and I lost my code"

**We suggest that `git` gives you
one way to overcome the
scenarios described before**

- Of course, not a magical solution
- Backups are still needed

Additional benefits

- Easier collaboration with other researchers
- Optionally, you can share your code
- Works for any text file (code, LaTeX, markdown etc.)

Limitations

- Not ideal for non-text files
 - .png, .docx, .pptx, binary files
- Not ideal for large individual files

Git vs. github/gitlab

- **Git** is a free and open source software
- Can be installed on your laptop
- Just using git is sufficient if all you want is to work on one machine by yourself
- github/gitlab/etc. are services that give users cloud/remote functionality based on git
- All TU/e users have access to **gitlab.tue.nl** for private repositories and to **github.com** where enterprise features can be requested.

European Git Hosting Services

Codeberg <https://codeberg.org> based on

Forgejo <https://forgejo.org/> (run your own Git server)

Configuring git

Git commits carry metadata about the author.
Two things you will always need to define somewhere
are:

```
git config --global user.name "Your Name"  
git config --global user.email yourname@example.com
```

Git sometimes needs to start a text editor for you to enter messages

(unless you create commits from inside an editor or on the web).

This may have already been set to something but if not **nano** is usually a safe choice:

```
git config --global core.editor nano
```

Create and clone a repo

To start, create a sample project in github.

1. In github, go to the Dashboard by clicking the github logo in the top left
2. In the left sidebar, at the top, select the **new** button
3. Select the **owner** (you) and type a **Repository name**
4. At **Configuration** Set : **Add README** to ON
5. Select **Create repository**.

Clone your repo

Now you can clone the repository in your project.

Cloning a repo means you're creating a copy on your computer, or wherever you want to store and work with the files.

Select your new Repository and make sure select the **Code** tab,
then copy the SSH URL from the dropdown button **Clone**

Open a terminal and go to the directory where you want to clone the files.

Run the command `git clone` with a pasted URL:

```
git clone git@github.com:username/my-sample-project.git
```

Go to the directory:

```
cd my-sample-project
```

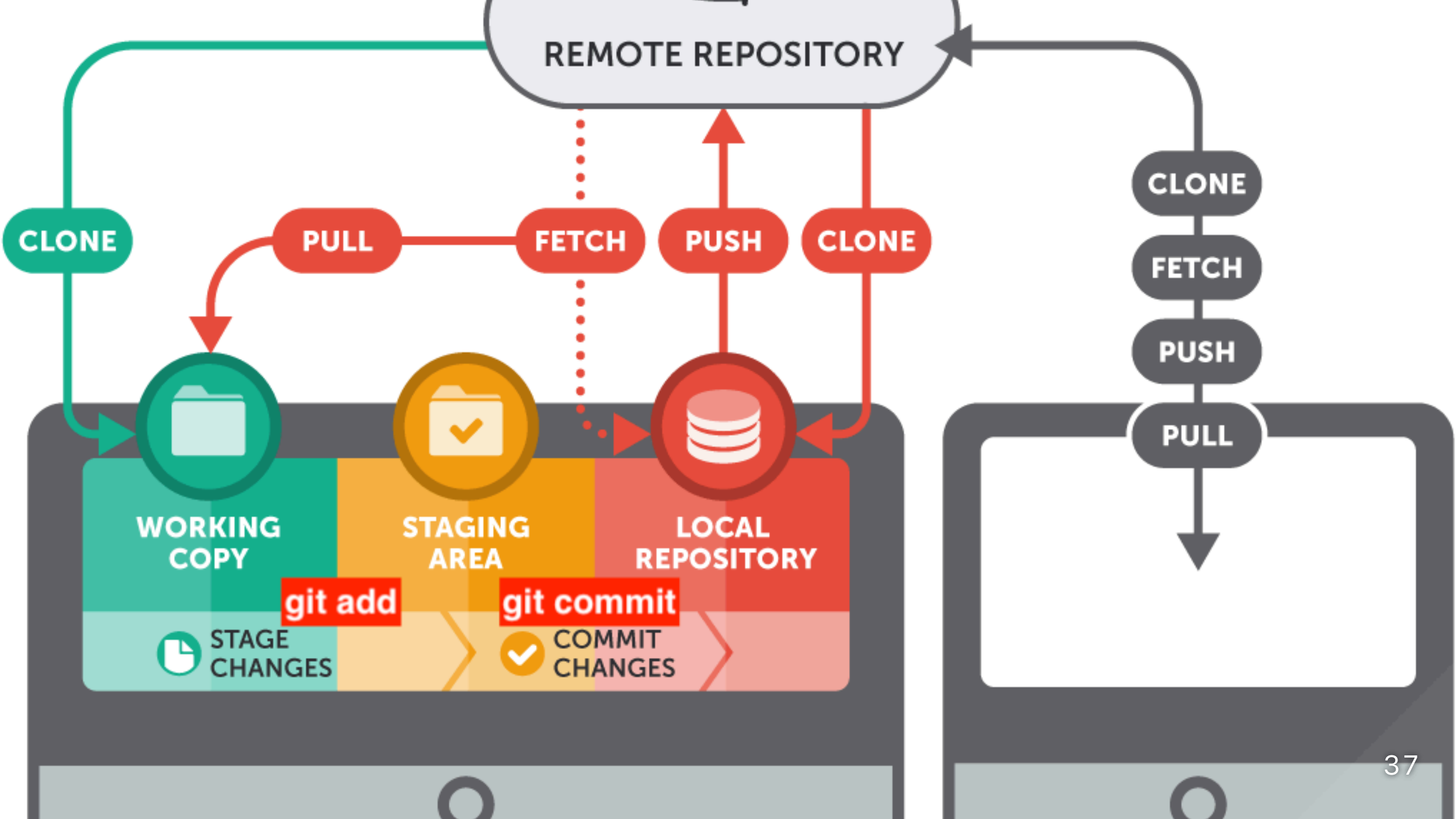
By default, you've cloned the default branch for the repository.

Usually this branch is `main`. To be sure, get the name of the default branch:

```
git branch
```

Using GIT

```
git clone  
git status  
git add  
git commit  
git push  
git pull
```



Make some changes

```
nano README.md  
git status
```

Commit the changes and Push them to the github repo

```
git add README.md  
git status  
git commit -m "Changed some lines"  
git status  
git push
```

<https://www.xkcd.org>

THIS IS GIT. IT TRACKS COLLABORATIVE WORK
ON PROJECTS THROUGH A BEAUTIFUL
DISTRIBUTED GRAPH THEORY TREE MODEL.

COOL. HOW DO WE USE IT?

NO IDEA. JUST MEMORIZE THESE SHELL
COMMANDS AND TYPE THEM TO SYNC UP.
IF YOU GET ERRORS, SAVE YOUR WORK
ELSEWHERE, DELETE THE PROJECT,
AND DOWNLOAD A FRESH COPY.



Look at the some history

```
git log  
git diff README.md
```

Exercise 1

1. In your local repo

Create 3 files: `secret.txt` `job1.sbatch` and `job2.sbatch`

2. Push files to the remote repo (github) but...

Make sure only `job1.sbatch` and `job2.sbatch` end up in the remote repo (github)

3. Check on github webinterface!

Exercise 2

Remove `job1.sbatch` from your local repo and the remote repo (github)

You will need the command:

```
git rm job1.sbatch
```

Use this to check "things" local

```
git status
```

`.gitignore`

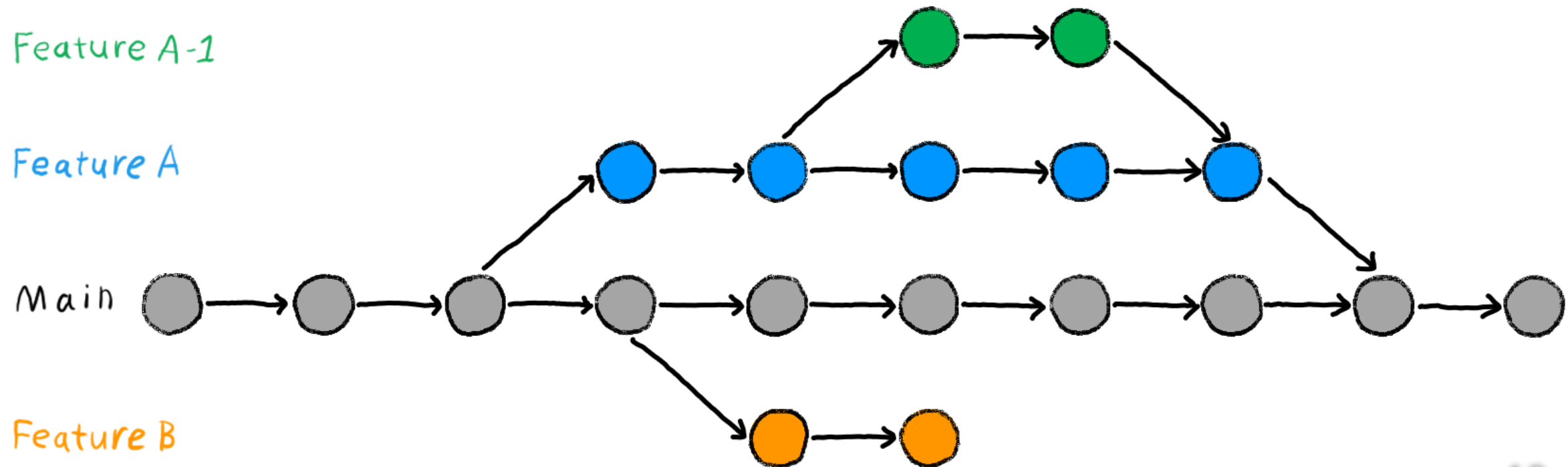
Filenames in the file `.gitignore`
are not considered a part of your repo by GIT.

Exercise 3

hide the file `secret.txt` from git

git branching vs. commit to main

If you're working with more than a few people on a code base using branches can be a good idea.



git branch

```
git branch mybranch  
git branch  
git switch mybranch  
nano README.md  
git add README.md  
git commit -m "Changed file in mybranch"  
git status  
git push --set-upstream origin mybranch
```

git merge request/pull request (PR)

Use the webinterface to create a PR and approve
the request (merge to main)

Finally delete the branch in the repo after a succesful
merge.

Now local:

```
git switch main  
git pull  
git branch -d mybranch # delete local
```

Exercise 4

1. Create a new branch : `NewBranch`
2. In that branch add a file `job3.sbatch`
3. Merge the branch to main
4. Check on github webinterface!

Exercise 5

Add a new directory to your local and the remote git repo

Exercise 6

Cleanup

1. Delete project from github
2. Delete project local

Suggested git workflow

- Use `git` to track:
 - code
 - input files
 - post-processing scripts
 - other critical (small) output files
- `git` is not the ideal tool to track large files
 - Other solutions are needed
 - Come talk to us if you are facing these issues

